Ex No: 6

6/8/2026

ERROR DETECTION & CORRECTION - HAMMING CODE

AIM: To write a program to implement error detection and correction using Hamming code.

ALGORITHM:

1. Start the program
2. Read the number of 4bit data blocks (n) forming the input data stream.
3. For each 4 bit data block $D = d_1\ d_2\ d_3\ d_4$, perform Hamming encoding.
   a) Arrange 4 bits with parity bits at position 1, 2 and 4, and data bits at positions 3, 5, 6 and 7.
   b) Compute the parity bits using even parity.
   $P_1 = d_1$ XOR d_2 XOR d_4 (covers bit positions 1, 3, 5, 7)
   $P_2 = d_1$ XOR d_3 XOR d_4 (covers bit positions 2, 3, 6, 7)
   $P_4 = d_2$ XOR d_3 XOR d_4 (covers bit positions 4, 5, 6, 7)
   c) Form the 7-bit Hamming code word:
   $P_1\ P_2\ d_1\ P_4\ d_2\ d_3\ d_4$
4. Simulate transmission by introducing a single bit error at a chosen bit position (entered by the user) in the code word.

5. At the receiver, recompute the check bits (syndrome) from the received code word.

$C_1 = P_1$ XOR d_1 XOR d_2 XOR d_4

$C_2 = P_2$ XOR d_1 XOR d_3 XOR d_4

$C_4 = P_4$ XOR d_2 XOR d_3 XOR d_4

6. Compute Error Position : $C_4\ C_2\ C_1$

7. If Error Position = 0, no error has occured Else, the bit at "Error Position" is inverted to correct the error.

8. Extract the original 4 data bits, from the corrected 7-bit code word.

9. Repeat steps 3 to 8 for all n-blocks in the data stream.

10. Stop the program.

PROGRAM:

```
def encode(d):
    """ d: list of 4bits [d1, d2, d3, d4] -> returns 7-bit
    hamming code list Positions (1-indexed):
    1=P1, 2=P2, 3=d1, 4=P4, 5=d2, 6=d3, 7=d4 """
    h = [0] * 7
    h[2] = d[0]  # position 3
    h[4] = d[1]  # position 5
    h[5] = d[2]  # position 6
    h[6] = d[3]  # position 7
```

```
    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
    h[3] = h[4] ^ h[5] ^ h[6]
    return h

def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]

    error_pos = c1 * 1 + c2 * 2 + c4 * 4

    if error_pos != 0:
        print(f" Error detected at bit position:
                {error_pos} -> Correcting ...")
        h[error_pos - 1] ^= 1
    else:
        print("No error detected.")
    return error_pos

def to_str(bits):
    return "".join(str(b) for b in bits)

def main():
    print("=== HAMMING CODE - ERROR
                DETECTION AND CORRECTION ===\n")
    n = int(input("Enter number of 4-bit data
                blocks in the stream: "))
```

OUTPUT:

=== HANNING CODE - ERROR DETECTION AND CORRECTION

Enter number of 4-bit data blocks in the stream: 2

--- Block 1 ---

Enter 4 data bits (space seperated e.g. 1011): 0 1 0 1

Original Data Bits: 0 1 0 1

Encoded Hamming code (7 bits): 0100101

Enter bit position to introduce error (1-7, 0=no error): 3

Transmitted Code (with error induced at pos 3): 0110101

Received Code: 0110101

Checking code: 0100101

Recovered Data bits: 0101

--- Block 2 ---

Enter 4 data bits (space seperated e.g. 1011): 1101

Original Data Bits: 1101

Encoded Hamming code (7 bits): 1010101

Enter bit position to introduce error (1-7, 0=no error): 5

Transmitted Code (with error induced at pos 5): 1010001

Received Code: 1010001

Checking for errors...

Error detected at bit position: 5 -> Correcting...

Corrected Code: 1010101

Recovered Code: 1101

```
for b in range(1, n+1):
    print(f"\n--- Block {b} ---")
    d = list(map(int, input("Enter 4 data bits
        (space separated, e.g. 1 0 1 1): ").split()))

    h = encode(d)
    print(f"Original Data bits : {to_str(d)}")
    print(f"Encoded Hamming Code (7 bits):
        {to_str(h)}")

    pos = int(input("Enter bit position to
        introduce error (1-7, 0 = no error): "))
    if 1 <= pos <= 7:
        h[pos-1] ^= 1
        print(f"Transmitted Code (with error
            induced at pos {pos}): {to_str(h)}")
    else:
        print("Transmitted Code (no error
            induced): {to_str(h)}")

    print(f"Received code : {to_str(h)}")
    print("Checking for errors...")
    detect_and_correct(h)
    print(f"Corrected code : {to_str(h)}")
    rd = [h[2], h[4], h[5], h[6]]
    print(f"Recovered Data bits : {to_str(rd)}")
```

```
if __name__ == "__main__":
    main()
```

RESULT:

The program to implement error detection & correction using hamming code concept was successfully written in python, executed and verified with a sample input data streams.